



University Institute of Engineering

Department of Computer Science & Engineering

Experiment: 9

Student Name: Aditya Raj Bhakta

UID:24BAI70782

Branch: CSE

Section/Group: AIT-KRG-G2

Semester: 4th

Date of Performance:17/04/2026

Subject Name: DBMS

1.Aim of the practical: To create and implement PL/SQL packages by developing a package specification and package body containing procedures and shared cursors, in order to achieve modular, reusable, and efficient database programming.

2. Tool Used:

- ☒ **Database Management System:**
- ☐ PostgreSQL
- ☒ **Database Administration Tool:**
- ☐ pgAdmin

3.Objective:

To design and implement a PL/SQL package that includes procedures and shared cursors for structured and modular program development.

4.Practical / Experimental Steps



University Institute of Engineering

Department of Computer Science & Engineering

Step 1 :Create the base table (Employee) with appropriate attributes such as employee ID, name, department, and salary.

Step 2 : Insert sample data into the table to simulate real-world employee records.

Step 3: Design a package-like structure by creating a schema (in PostgreSQL) or package specification (in Oracle) to declare procedures/functions.

Step 4: Implement procedures/functions to fetch and display employee details.

Step 5: Define and use a shared cursor (or cursor logic) to retrieve employee records efficiently within the procedures.

Step 6: Execute the created procedures/functions to display all employee records and filtered results (e.g., by department).

5. I / O Analysis

```
CREATE TABLE Employee (  
  emp_id INTEGER PRIMARY KEY,  
  emp_name VARCHAR(50),  
  department VARCHAR(50),  
  salary NUMERIC  
);
```

```
CREATE TABLE
```

```
Query returned successfully in 126 msec.
```

```
INSERT INTO Employee VALUES (101, 'Amit', 'HR', 30000);  
INSERT INTO Employee VALUES (102, 'Riya', 'IT', 50000);  
INSERT INTO Employee VALUES (103, 'Karan', 'Finance', 45000);  
INSERT INTO Employee VALUES (104, 'Neha', 'IT', 55000);
```

```
INSERT 0 1
```

```
Query returned successfully in 45 msec.
```

```
CREATE SCHEMA emp_package;
```



University Institute of Engineering

Department of Computer Science & Engineering

```
CREATE SCHEMA
```

```
Query returned successfully in 37 msec.
```

```
CREATE OR REPLACE FUNCTION emp_package.get_all_employees()
RETURNS VOID AS $$
DECLARE
    emp_rec RECORD;
BEGIN
    FOR emp_rec IN
        SELECT * FROM employee
    LOOP
        RAISE NOTICE '% % % %',
            emp_rec.emp_id,
            emp_rec.emp_name,
            emp_rec.department,
            emp_rec.salary;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION
```

```
Query returned successfully in 109 msec.
```

```
CREATE OR REPLACE FUNCTION emp_package.get_emp_by_dept(p_dept VARCHAR)
RETURNS VOID AS $$
DECLARE
    emp_rec RECORD;
BEGIN
    FOR emp_rec IN
        SELECT * FROM employee
    LOOP
        IF emp_rec.department = p_dept THEN
            RAISE NOTICE '% % % %',
                emp_rec.emp_id,
                emp_rec.emp_name,
                emp_rec.department,
                emp_rec.salary;
        END IF;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```



University Institute of Engineering

Department of Computer Science & Engineering

```
CREATE FUNCTION
```

```
Query returned successfully in 38 msec.
```

OUTPUT

```
SELECT emp_package.get_all_employees();
```

```
NOTICE: 101 Amit HR 30000
```

```
NOTICE: 102 Riya IT 50000
```

```
NOTICE: 103 Karan Finance 45000
```

```
NOTICE: 104 Neha IT 55000
```

```
Successfully run. Total query runtime: 84 msec.
```

```
1 rows affected.
```

```
SELECT emp_package.get_emp_by_dept('IT');
```

```
NOTICE: 102 Riya IT 50000
```

```
NOTICE: 104 Neha IT 55000
```

```
Successfully run. Total query runtime: 62 msec.
```

```
1 rows affected.
```

6. Learning outcomes (What I have learnt):

- ☐ Understand the concept and working of materialized views in a database system.
- ☐ Differentiate between simple views, complex views, and materialized views.
- ☐ Create and refresh materialized views in PostgreSQL.
- ☐ Measure and compare query execution time for different types of views.
- ☐ Analyze performance benefits of materialized views in data-intensive applications.
- ☐ Apply materialized view concepts in real-world company scenarios such as SanDisk, JTG, and PayPal.